

Does Random Mean Secure?

A probability and statistics project proposal

tdvnm

September 17, 2026

The problem

Encryption needs a key that an attacker cannot predict.

Same encryption rule. Different key sources. Different security.

The question

If the key becomes more predictable, how much more does the ciphertext tell us about the message?

The route through the project

1. **Test the source.** Do its bits behave like independent fair coin flips?
2. **Use it as a key.** Encrypt with a simple XOR cipher.
3. **Measure the leak.** How much does the ciphertext tell us?
4. **Check the limit of tests.** Can output look random while its future is predictable?

The cipher and message distribution stay fixed in the main experiment. Only the key source changes.

What would an ideal bit source do?

Let $B_1, \dots, B_n \in \{0, 1\}$ be the output bits.

Statistical model

Under H_0 , B_i are independent and each is $\text{Bern}(1/2)$. Every n -bit sequence has probability 2^{-n} .

Two simple ways to violate this model:

- **Bias:** ones appear more often than zeros.
- **Dependence:** a bit depends on the bit before it.

A good test for one failure may miss the other.

Two tests, two different questions

Test	Statistic	Under independent fair bits
Frequency	$X = \sum_{i=1}^n B_i$ (number of ones)	$X \sim \text{Bin}(n, 1/2)$
Runs	$R = 1 + \sum_{i=1}^{n-1} 1\{B_i \neq B_{i+1}\}$	$R - 1 \sim \text{Bin}(n - 1, 1/2)$

A biased coin should affect frequency. A source that switches too often can have about half ones and still fail the runs test.

What a test result actually says

A p -value asks: *if the ideal model were true, how unusual would this test statistic be?*

- A small p -value is evidence against that model.
- A large p -value is not proof that the source is unpredictable.
- These tests examine the output, not what an attacker knows about the source.

Planned comparison: fair bits, biased bits, a source with too many switches, and a simple deterministic generator.

The gap: random-looking versus unpredictable

A pseudorandom generator begins with a state S_0 and computes

$$S_{i+1} = f(S_i), \quad B_i = g(S_i).$$

Its output may pass the two tests on a finite sample. But if an attacker recovers a small state, the future output is fixed.

Why the distinction matters

A key source must resist prediction by an attacker; passing a few output tests does not establish that property.

The encryption model

Use two-bit messages and keys:

$$C = M \oplus K.$$

For example, $01 \oplus 10 = 11$. The receiver recovers the message because $C \oplus K = M$.

Attacker's view

The attacker sees C and knows the encryption rule and the distributions of M and K , but not the chosen key.

Assume the key is independent of the message and used once.

What perfect secrecy means

Seeing the ciphertext should leave beliefs about the message unchanged:

$$\mathbb{P}(M = m \mid C = c) = \mathbb{P}(M = m).$$

With a uniform two-bit key, each key has probability $1/4$, so

$$\mathbb{P}(C = c \mid M = m) = \mathbb{P}(K = m \oplus c) = \frac{1}{4}$$

for every m, c . The ciphertext therefore gives no clue about which message was selected. This is the one-time pad argument for this small model.

A number for information leakage

Entropy measures uncertainty. Conditional entropy is what remains after seeing the ciphertext.

$$I(M; C) = H(M) - H(M | C)$$

- $I(M; C) = 0$: ciphertext tells us nothing about the message.
- Larger $I(M; C)$: the ciphertext reveals more.
- The most it can reveal is $H(M)$.

The main experiment: change only the key

Fixed

- $\mathbb{P}(M = 00, 01, 10, 11) = (0.4, 0.3, 0.2, 0.1)$
- $C = M \oplus K$
- Key independent of the message

Changed

- K_1, K_2 independent $\text{Bern}(q)$
- $q = 0.50, 0.55, \dots, 0.95$
- $H(K) = 2h(q)$ bits

At $q = 0.50$, all four keys are equally likely. As q grows, the key 11 becomes increasingly likely and the key is easier to guess.

How I calculate the exact answer

For each ciphertext c , sum over the four possible messages:

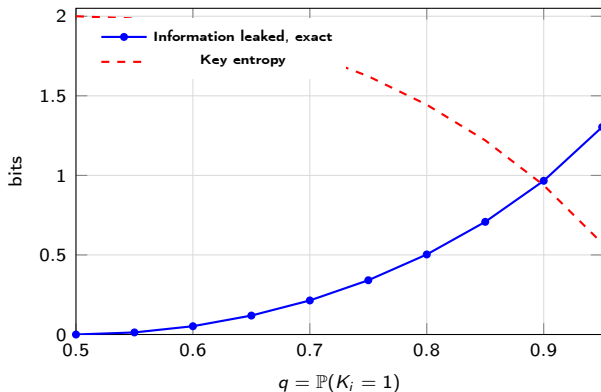
$$\mathbb{P}(C = c) = \sum_m \mathbb{P}(M = m) \mathbb{P}(K = m \oplus c).$$

Because XOR with a fixed message only rearranges the keys, $H(C | M) = H(K)$. Hence

$$\boxed{I(M; C) = H(C) - H(K)}.$$

Four messages and four keys make the calculation exact and checkable.

Calculated result: weaker key, more leakage



At $q = 0.50$: $I(M; C) = 0$.

At $q = 0.95$: key entropy is 0.57 bits and leakage is 1.30 bits, out of $H(M) \approx 1.85$ bits.

These are **exact calculations**; the simulation is planned.

How I will check the calculation

At each value of q :

1. Draw 10,000 independent message–key pairs.
2. Encrypt them and count the (M, C) pairs.
3. Estimate $I(M; C)$ from the observed frequencies.
4. Repeat the simulation and compare its spread with the exact curve.

A finite-sample leakage estimate may be slightly above zero even when the exact leakage is zero. Repeating at larger sample sizes will show whether the estimate approaches the exact answer.

What this project can and cannot establish

- The exact curve answers the question for this two-bit XOR model.
- The simulation checks the calculation and the estimator.
- Frequency and runs tests detect specific failures of a bit source.
- Passing those tests alone does not prove cryptographic security.

A two-bit cipher makes the probability visible. A real key generator needs a separate analysis of whether an attacker can predict it.

Takeaway

As the key becomes easier to guess, the ciphertext can reveal more about the message.

Statistical tests help identify bad randomness.
They do not, by themselves, establish secure randomness.

Next: compare simulated leakage with the exact curve.

References and backup

- C. E. Shannon (1949), *Communication Theory of Secrecy Systems*: perfect secrecy.
- C. E. Shannon (1948), *A Mathematical Theory of Communication*: entropy and information.
- NIST SP 800-22 Rev. 1a (2010): statistical tests for bit generators.

Backup examples: Caesar cipher, human-generated sequences, and recovering a small generator state are in the longer `main.pdf` deck.